

Algoritmul lui Euclid | Exponențiere rapidă

Andrei Mihai Ciobanu

Algolymp Summer School

Cuprins

Algoritmul lui Euclid

Algoritmul lui Euclid extins

Exponențiere rapidă

Bonus — Implementări recursive

Recapitulare

Algoritmul lui Euclid

Cel mai mare divizor comun

Cel mai mare divizor comun

cmmdc(a, b) = cel mai mare număr natural d astfel încât $d \mid a$ și $d \mid b$. Se mai notează și gcd(a, b) (*greatest common divisor*) sau doar (a, b).

Cel mai mic multiplu comun

cmmmc(a, b) = cel mai mic număr natural m astfel încât $a \mid m$ și $b \mid m$. Se mai notează și lcm(a, b) (*lowest common multiple*) sau doar $[a, b]$.

Exemple:

- $\gcd(12, 8) = 4$ $\gcd(17, 5) = 1$ $\gcd(100, 75) = 25$
- $\text{lcm}(4, 6) = 12$ $\text{lcm}(3, 5) = 15$

Calculul cmmdc

Întrebare

Cum se calculează $\gcd(84, 36)$? Dar $\gcd(10^{18},\ 6 \cdot 10^{17})$?

- Se listează divizorii lui a în $O(\sqrt{a})$ și se testează fiecare pe b. Pentru $\gcd(84, 36) = 12$ metoda funcționează.
- Pentru numere de ordinul 10^{18} rezultă circa 10^9 operații — prea lent.

Necesitatea unei metode mai rapide

Algoritmul lui Euclid: sub 100 de pași pentru numere de 10^{18}.

Ideea de bază: $\gcd(a, b) = \gcd(a - b,\ b)$

Întrebare

Fie d un divizor comun al lui a și b. Ce se poate afirma despre $a - b$?

Hint

Se scriu $a = d \cdot x$ și $b = d \cdot y$, apoi se calculează $a - b$.

- $a - b = d \cdot (x - y)$, deci $d \mid a - b$.
- Reciproc, dacă $d \mid a - b$ și $d \mid b$, atunci $d \mid (a - b) + b = a$.

Concluzie

Perechile (a, b) și $(a - b,\ b)$ au **aceiași divizori comuni**, deci $\gcd(a, b) = \gcd(a - b,\ b)$.

Scăderi repetate

Regula $\gcd(a, b) = \gcd(a - b, b)$ se aplică repetat până la cazul de bază $\gcd(a, 0) = a$.

Întrebare

Cât este $\gcd(12, 8)$ dacă se folosesc **doar** scăderi?

$$\gcd(12, 8) = \gcd(4, 8) = \gcd(4, 4) = \gcd(0, 4) = 4 \quad \checkmark$$

Întrebare

Câți pași sunt necesari pentru $\gcd(1000, 1)$?

$$\gcd(1000, 1) \to \gcd(999, 1) \to \gcd(998, 1) \to \cdots \to \gcd(0, 1) = 1$$

1000 de pași pentru un rezultat evident — procedeul este prea lent pentru valori mari.

De la scăderi la rest

Întrebare

Cum pot fi înlocuite scăderile repetate ale lui b cu o singură operație?

Prin împărțirea cu rest, $a = q \cdot b + r$, cu $0 \leq r < b$. Restul r se notează $a \bmod b$ (restul împărțirii lui a la b):

$$\gcd(a, b) = \gcd(a - b, b) = \gcd(a - 2b, b) = \cdots = \gcd(a \bmod b,\ b)$$

Algoritmul lui Euclid

$\gcd(a, b) = \gcd(b,\ a \bmod b)$, cu cazul de bază $\gcd(a, 0) = a$.

Exemplu: $\gcd(48, 18)$

- $\gcd(48, 18) = \gcd(18, 12)$ $(48 \bmod 18 = 12)$
- $\gcd(18, 12) = \gcd(12, 6)$ $(18 \bmod 12 = 6)$
- $\gcd(12,\ 6) = \gcd(6,\ 0)$ $(12 \bmod 6 = 0)$
- $\gcd(6, 0) = 6$ ✓

Proprietăți

- $\gcd(a, 0) = \gcd(a, a) = |a|$ (*cazuri de bază*)
- $\gcd(a, b) = \gcd(b, a)$ (*comutativitate*)
- $\gcd(a, \gcd(b, c)) = \gcd(\gcd(a, b), c)$ (*asociativitate*)
- $\gcd(k \cdot a, k \cdot b) = k \cdot \gcd(a, b)$, pentru $k > 0$
- dacă $\gcd(a, b) = d$, atunci $\gcd\left(\frac{a}{d}, \frac{b}{d}\right) = 1$
- $a \cdot b = \gcd(a, b) \cdot \text{lcm}(a, b)$
- dacă $\gcd(a, b) = 1$, se spune că a și b sunt *prime între ele* (*coprime*)
- $\gcd(a, b) = \gcd(a \pm k \cdot b,\ b),\ \forall\, k \in \mathbb{N} \quad \Rightarrow \quad \gcd(a, b) = \gcd(b,\ a \bmod b)$
- **Prin descompunere în factori primi:** dacă $n = \prod p_i^{a_i}$ și $m = \prod p_i^{b_i}$, atunci $\gcd(n, m) = \prod p_i^{\min(a_i,\, b_i)}$ și $\text{lcm}(n, m) = \prod p_i^{\max(a_i,\, b_i)}$
  Exemplu: $n = 2^3 \cdot 3^2,\ m = 2^1 \cdot 3^4 \ \Rightarrow\ \gcd = 2^1 \cdot 3^2 = 18,\ \text{lcm} = 2^3 \cdot 3^4 = 648$

Algoritmul lui Euclid — implementare iterativă

Explicația implementării

- Se pornește de la $\gcd(a, b) = \gcd(a \bmod b,\ b) = \gcd(b,\ a \bmod b)$
- Cât timp $b > 0$, se calculează restul $r = a \bmod b$; apoi noul a devine b, iar noul b devine r
- După fiecare pas, noul b este mai mic decât noul a ($r < b$), deci primul termen rămâne cel mai mare și operația $a \bmod b$ se poate efectua în continuare
- Dacă inițial $a < b$, atunci $a \bmod b = a$, iar primul pas doar interschimbă valorile: $(a, b) \to (b, a)$
- Când $b = 0$, rezultatul este a (cazul de bază $\gcd(a, 0) = a$)

```
1 int gcd(int a, int b) {
2     while (b != 0) {
3         int r = a % b;
4         a = b;
5         b = r;
6     }
7     return a;
8 }
```

- Timp: $O(\log \min(a, b))$ Spațiu: $O(1)$

Cel mai mic multiplu comun — calcul

Întrebare

Cum se calculează $\text{lcm}(a, b)$ dacă se cunoaște $\gcd(a, b)$?

Din $a \cdot b = \gcd(a, b) \cdot \text{lcm}(a, b)$ rezultă `lcm = (a / gcd(a, b)) * b`.

Ordinea operațiilor

Este indicat ca împărțirea să se facă prima: $a \cdot b$ poate produce overflow chiar dacă $\text{lcm}(a, b)$ încape în tipul de date, iar astfel rezultatul rămâne corect.

Complexitatea algoritmului lui Euclid

Întrebare

Cât devine a după **doi** pași ai algoritmului?

Hint

Se separă cazurile $b \leq a/2$ și $b > a/2$. Cât este $a \bmod b$ în fiecare caz?

- dacă $b \leq a/2$: $a \bmod b < b \leq a/2$
- dacă $b > a/2$: $a \bmod b = a - b < a/2$

Concluzie

În ambele cazuri $a \bmod b < a/2$, deci după **doi** pași cel mai mare număr se înjumătățește cel puțin $\Rightarrow O(\log \min(a, b))$ pași.

▷ log = *numărul de înjumătățiri succesive până când un număr ajunge la* 1; $\log_2 10^{18} \approx 60$

Algoritmul lui Euclid extins

Identitatea lui Bézout

Identitatea lui Bézout

Pentru orice $a, b \in \mathbb{Z}$, există $x, y \in \mathbb{Z}$ astfel încât:

$$a \cdot x + b \cdot y = \gcd(a, b)$$

Exemplu: $22 \cdot 3 + 13 \cdot (-5) = 66 - 65 = 1 = \gcd(22, 13)$.

Întrebare

Cum se determină și numerele x, y, nu doar gcd?

Algoritmul lui Euclid reduce (a, b) la o pereche mai mică, $(b,\ a \bmod b)$. Soluția pentru perechea mai mică poate fi folosită pentru perechea inițială?

▷ *Secțiunea conține un concept ușor mai dificil — poate fi reluată după parcurgerea restului lecției.*

Euclid extins — ideea

Fie $g = \gcd(a, b)$ și ecuația $a \cdot x + b \cdot y = g$. Se scrie $a = q \cdot b + r$, unde $q = \lfloor a/b \rfloor$ și $r = a \bmod b$.

Întrebare

Ce formă are ecuația după înlocuirea lui a și gruparea termenilor care conțin b?

$$\underbrace{(q \cdot b + r)}_{a} \cdot x + b \cdot y = g \quad \Rightarrow \quad b \cdot (q \cdot x + y) + r \cdot x = g$$

Cu notațiile $x' = q \cdot x + y$ și $y' = x$, ecuația devine $b \cdot x' + r \cdot y' = g$.

- Forma este aceeași, cu $(b,\ r)$ în loc de $(a,\ b)$ — exact un pas din algoritmul lui Euclid.
- Reducerea continuă până la cazul de bază $b = 0$: $g \cdot x + 0 \cdot y = g \ \Rightarrow \ x = 1,\ y = 0$

Reconstrucție la întoarcere

Din $x' = q \cdot x + y$ și $y' = x$ rezultă $x = y', \quad y = x' - q \cdot y'$.

Exemplu pe hârtie: $22x + 13y = \gcd(22, 13)$

Pasul 1: Se aplică algoritmul lui Euclid

- $22 = 1 \cdot 13 + 9$ $\Rightarrow 9 = 22 - 1 \cdot 13$
- $13 = 1 \cdot 9 + 4$ $\Rightarrow 4 = 13 - 1 \cdot 9$
- $9 = 2 \cdot 4 + 1$ $\Rightarrow 1 = 9 - 2 \cdot 4$
- $4 = 4 \cdot 1 + 0 \;\Rightarrow\; \gcd(22, 13) = 1$

Întrebare

Cum se exprimă 1 folosind doar 22 și 13?

Pasul 2: Substituție în ordine inversă, de jos în sus

- $1 = 1 \cdot \mathbf{9} - 2 \cdot \mathbf{4}$
- $1 = 1 \cdot \mathbf{9} - 2 \cdot (\mathbf{13} - 1 \cdot \mathbf{9}) = 3 \cdot \mathbf{9} - 2 \cdot \mathbf{13}$
- $1 = 3 \cdot (1 \cdot \mathbf{22} - 1 \cdot \mathbf{13}) - 2 \cdot \mathbf{13} = 3 \cdot \mathbf{22} - 5 \cdot \mathbf{13}$

Soluție

$x = 3,\ y = -5 \quad \checkmark \quad (22 \cdot 3 + 13 \cdot (-5) = 66 - 65 = 1)$

Euclid extins — implementare iterativă

```
1  // Gaseste x, y cu: a*x + b*y = gcd(a, b)
2  int gcd_ext(int a, int b, int &x, int &y) {
3      int x0 = 1, y0 = 0;
4      int x1 = 0, y1 = 1;
5      while (b != 0) {
6          int q = a / b;
7          int xn = x0 - q * x1;
8          int yn = y0 - q * y1;
9          int r = a % b;
10         a = b; b = r;
11         x0 = x1; y0 = y1;
12         x1 = xn; y1 = yn;
13     }
14     x = x0; y = y0;
15     return a;
16 }
```

- `int &x` = parametru transmis prin referință: modificările din funcție sunt vizibile în apelant
- Timp: $O(\log \min(a, b))$ Spațiu: $O(1)$

Inițializarea coeficienților x_0, y_0, x_1, y_1

Fie a_0, b_0 valorile inițiale. La orice pas al buclei sunt menținute relațiile:

$$a = a_0 \cdot x_0 + b_0 \cdot y_0 \qquad b = a_0 \cdot x_1 + b_0 \cdot y_1$$

Întrebare

Ce valori trebuie să aibă x_0, y_0, x_1, y_1 înainte de primul pas?

- Înainte de primul pas, $a = a_0$ și $b = b_0$.
- $a_0 = a_0 \cdot 1 + b_0 \cdot 0$ și $b_0 = a_0 \cdot 0 + b_0 \cdot 1$, deci $x_0 = 1,\ y_0 = 0,\ x_1 = 0,\ y_1 = 1$.
- La final $b = 0$ și $a = \gcd$, deci x_0, y_0 sunt coeficienții căutați.

Exponențiere rapidă

Exponențiere rapidă — ideea

Problema

Calculul eficient al lui a^n. Varianta naivă (n înmulțiri) are complexitate $O(n)$.

Întrebare

Cum se poate calcula 2^{10} cu mai puțin de 9 înmulțiri?

- $2^{10} = (2^5)^2$: 2^5 se calculează o singură dată și se ridică la pătrat
- Aceeași jumătate nu se mai calculează de două ori: exponentul se înjumătățește

Întrebare

Cum se procedează dacă exponentul este impar, de exemplu 2^5?

- Se scoate o bază în afară: $2^5 = 2 \cdot 2^4$, iar 2^4 are exponent par

$$a^n = \begin{cases} 1 & n = 0 \\ \left(a^{n/2}\right)^2 & n \text{ par} \\ a \cdot a^{n-1} & n \text{ impar} \end{cases}$$

Numărul de pași

Exemplu: $2^{10} \to 2^5 \to 2^4 \to 2^2 \to 2^1 \to 2^0$ (5 pași în loc de 10)

Întrebare

Câți pași sunt necesari pentru $n = 10^9$?

- 10^9 se împarte la 2 de circa 30 de ori, deoarece $2^{30} \approx 10^9$
- La fiecare înjumătățire se face cel mult o înmulțire suplimentară pentru exponentul impar, deci cel mult circa $2 \cdot 30$ înmulțiri

Concluzie

Complexitate $O(\log n)$, în loc de 10^9 pași.

Exponențiere rapidă — implementare iterativă

```cpp
long long power(long long base, int exp) {
    long long result = 1;
    while (exp > 0) {
        if (exp % 2 == 1)
            result *= base;
        base *= base;
        exp /= 2;
    }
    return result;
}
```

- Timp: $O(\log n)$ Spațiu: $O(1)$
- Atenție: `base * base` poate da overflow pentru baze mari — în concursuri se lucrează de obicei modulo un număr prim

Urmărirea execuției: `power(3, 5)`

Întrebare

Ce valori au `result`, `base` și `exp` la finalul fiecărei iterații?

Iterația	`result`	`base`	`exp`
început	1	3	5
1 (`exp` = 5, impar)	3	9	2
2 (`exp` = 2, par)	3	81	1
3 (`exp` = 1, impar)	243	6561	0

Rezultat

`power(3, 5)` $= 243 = 3^5$ ✓

Legătura cu scrierea binară

În exemplul `power(3, 5)`: $5 = 101_{(2)}$, iar `base` a luat valorile 3^1, 3^2, 3^4.

Întrebare

Ce legătură există între algoritm și scrierea lui n în baza 2?

Hint

Se descompune $5 = 4 + 1$ și se scrie 3^5 ca produs de puteri ale lui 3.

- $3^5 = 3^4 \cdot 3^1$; `base` parcurge $a^{2^0}, a^{2^1}, a^{2^2}, \ldots$, iar `result` se înmulțește doar cu bazele corespunzătoare **biților egali cu** 1 din scrierea binară a lui n

Concluzie

Numărul de biți ai lui n este $\approx \log_2 n$, deci $O(\log n)$ iterații.

Bonus — Implementări recursive

Definiție

O funcție este **recursivă** dacă se apelează pe ea însăși.

Orice funcție recursivă are nevoie de:

- Un **caz de bază** — condiția de oprire
- Un **apel recursiv** pe o subproblemă mai mică

Exemplu: $n! = n \cdot (n-1)!$, cu cazul de bază $0! = 1$

Întrebare

Ce se întâmplă dacă lipsește cazul de bază?

- Funcția s-ar apela la nesfârșit până se umple stiva de apeluri — **stack overflow**

Atenție — dezavantajele recursivității

Overhead de apeluri de funcție (ușor mai lent) și risc de **stack overflow** pentru intrări foarte mari. Codul recursiv reflectă direct definiția matematică, dar în concursuri este preferată varianta iterativă.

Euclid — implementare recursivă

Întrebare

Cum se scrie gcd recursiv pornind de la $\gcd(a, b) = \gcd(b,\ a \bmod b)$? Care este cazul de bază?

```
int gcd(int a, int b) {
    if (b == 0)
        return a;
    return gcd(b, a % b);
}
```

- Cazul de bază este $\gcd(a, 0) = a$
- Codul reflectă direct proprietatea $\gcd(a, b) = \gcd(b,\ a \bmod b)$
- Aceeași complexitate ca varianta iterativă: $O(\log \min(a, b))$

Euclid extins — implementare recursivă

Întrebare

Ce valori au x și y în cazul de bază $b = 0$? Cum se reconstruiesc la întoarcere?

- Cazul de bază: $a \cdot x + 0 \cdot y = a$, deci $x = 1,\ y = 0$
- La întoarcere: $x = y',\quad y = x' - q \cdot y'$

```
1  int gcd_ext(int a, int b, int &x, int &y) {
2      if (b == 0) {
3          x = 1; y = 0;
4          return a;
5      }
6      int x1, y1;
7      int d = gcd_ext(b, a % b, x1, y1);
8      x = y1;
9      y = x1 - (a / b) * y1;
10     return d;
11 }
```

- Timp: $O(\log \min(a, b))$ Spațiu: $O(\log \min(a, b))$ — stiva de apeluri

Exponențiere rapidă — implementare recursivă

```
1 long long power(long long base, int exp) {
2     if (exp == 0)
3         return 1;
4     if (exp % 2 == 0) {
5         long long half = power(base, exp / 2);
6         return half * half;
7     }
8     return base * power(base, exp - 1);
9 }
```

- Timp: $O(\log n)$ Spațiu: $O(\log n)$ — stiva de apeluri

Întrebare

De ce nu se scrie direct `return power(base, exp / 2) * power(base, exp / 2);`?

- Două apeluri la fiecare nivel $\Rightarrow$ numărul de apeluri se dublează la fiecare nivel
- În total $O(n)$ apeluri — câștigul se pierde; cu `half * half` se face **un singur** apel pe nivel: $O(\log n)$

Recapitulare

Recapitulare — Complexități

Algoritm	Timp	Spațiu
gcd iterativ	$O(\log \min(a, b))$	$O(1)$
gcd extins iterativ	$O(\log \min(a, b))$	$O(1)$
gcd recursiv	$O(\log \min(a, b))$	$O(\log \min(a, b))$
gcd extins recursiv	$O(\log \min(a, b))$	$O(\log \min(a, b))$
Exponențiere rapidă iterativă	$O(\log n)$	$O(1)$
Exponențiere rapidă recursivă	$O(\log n)$	$O(\log n)$

Probleme de antrenament

AlgoCode

1. **GCD and LCM** (AlgoCode #1557)
2. **GCD and LCM Queries** (AlgoCode #4230)
3. **GCD Power Queries** (AlgoCode #4270)

Altele

1. **cmmdc** (pbinfo.ro)
2. **cmmmc** (pbinfo.ro)
3. **euclid3** (infoarena.ro)
4. **lgput** (infoarena.ro)

Resurse

1. **infoarena.ro** — Algoritmul lui Euclid (RO)
2. **cp-algorithms.com** — Euclidean algorithm (EN)
3. **cp-algorithms.com** — Extended Euclidean algorithm (EN)
4. **cp-algorithms.com** — Binary exponentiation (EN)
5. **pbinfo.ro** — Recursivitate (RO)
6. **cppreference.com** — documentație completă C++